

Interview Preparation — PostgreSQL & Database Engineering

A complete guide to navigating technical interviews for roles that require PostgreSQL, SQL, or database engineering expertise. Covers SQL rounds, internals interviews, system design, behavioral questions, salary negotiation, and company-specific preparation.

Table of Contents

1. [How Interviews Are Structured](#)
 2. [The SQL Interview Round](#)
 3. [Explaining Query Optimization Under Pressure](#)
 4. [How to Talk About Internals](#)
 5. [System Design with Databases](#)
 6. [Behavioral Questions for Database Engineers](#)
 7. [Level-Specific Question Banks](#)
 8. [The Salary Negotiation Playbook](#)
 9. [Company-Specific Preparation](#)
 10. [The Day-Before Checklist](#)
 11. [Post-Interview Protocol](#)
-

1. How Interviews Are Structured

The Typical Interview Funnel for DB-Heavy Roles

```
Recruiter Screen (30 min)
├─ Technical Phone Screen (45-60 min)
│   └─ Take-Home Assignment (optional, 2-4 hours)
│       └─ On-Site / Virtual On-Site Loop (4-6 rounds)
│           ├── SQL / Coding Round (45-60 min)
│           ├── System Design Round (60 min)
│           ├── Internals / Deep Dive Round (45-60 min) [Senior+]
│           ├── Behavioral / Leadership Round (45 min)
│           └─ Hiring Manager Round (30-45 min)
```

What Each Round Tests

SQL / Coding Round: Can you write correct, efficient SQL for real-world problems? Do you handle edge cases (NULLs, duplicates, empty sets)? Can you think through complexity trade-offs?

System Design Round: Can you design a database-backed system at scale? Do you know when to use read replicas, caching, partitioning, and when to reach for a different database? Can you reason about CAP theorem trade-offs?

Internals / Deep Dive Round: This round exists at senior+ levels. It separates engineers who use PostgreSQL from engineers who understand PostgreSQL. Topics: MVCC, WAL, VACUUM, replication, query planning.

Behavioral Round: Past behavior predicts future behavior. They want to see how you handle incidents, disagreements, and complex multi-stakeholder situations — with databases.

Role-Specific Interview Emphasis

Role	SQL	System Design	Internals	Behavioral
Data Analyst	Very High	Low	Low	Low
Analytics Engineer	Very High	Medium	Low	Low
Data Engineer	High	High	Medium	Medium
Backend Engineer	High	High	Medium	Medium
Database Engineer / DBA	High	High	Very High	Medium
Staff Engineer	Medium	Very High	High	Very High

2. The SQL Interview Round

What Interviewers Are Actually Looking For

They are not just checking if you know syntax. They are watching for:

1. **Problem decomposition** — Do you break the problem down before writing a single line?
2. **Edge case awareness** — Do you ask about NULLs, duplicates, empty tables?
3. **Communication** — Do you narrate your thinking or just silently type?
4. **Iteration** — Can you refine your query when given a follow-up constraint?
5. **Correctness vs performance** — Do you know when "correct but slow" is acceptable to submit vs when optimization matters?

The 5-Step Framework for Any SQL Problem

Step 1 — Read and Restate (1 minute) Read the problem. Restate it in your own words out loud. "So we need to find the top 3 customers by revenue per region in the last 30 days, including ties. Is that correct?"

Step 2 — Ask Clarifying Questions (1–2 minutes)

- "Can revenue be NULL or negative?"
- "Should I include customers with zero revenue, or only those with at least one order?"
- "For 'last 30 days' — is that calendar days or rolling 30 days from today?"
- "Do we want ties? If two customers have the same revenue, should both appear?"

Step 3 — Plan Your Approach (2 minutes) Say your approach before writing. "I'm going to use a CTE to compute revenue per customer per region, then use RANK() with PARTITION BY region to rank them, then filter for rank <= 3 in the outer query."

Step 4 — Write the Query (5–10 minutes) Write in stages. Start with the simplest CTE. Add the window function. Add the filter. Test each stage conceptually.

Step 5 — Review and Verify (2 minutes) Walk through the query with a sample of 3–5 rows in your head. "For a customer with no orders, this CTE would return NULL revenue. We should handle that with a COALESCE or a different join type."

Common SQL Interview Question Categories

Category 1: Window Functions (Highest Frequency)

These appear in virtually every data/analytics/backend SQL screen.

Classic patterns to know cold:

```
-- 1. Running total
SELECT
    date,
    revenue,
    SUM(revenue) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING) AS running_total
FROM daily_revenue;

-- 2. Percentage of total
SELECT
    category,
    revenue,
    revenue / SUM(revenue) OVER () * 100 AS pct_of_total
FROM category_revenue;

-- 3. Row-within-group ranking (deduplication)
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) AS rn
    FROM user_events
)
SELECT * FROM ranked WHERE rn = 1; -- latest event per user

-- 4. Month-over-month growth
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS change,
    round(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS pct_change
FROM monthly_revenue;

-- 5. Percentile assignment
SELECT
    user_id,
    total_spend,
    NTILE(10) OVER (ORDER BY total_spend DESC) AS spend_decile
FROM user_totals;
```

Category 2: Aggregation with Filtering

```
-- Conditional aggregation (replace pivots)
SELECT
    customer_id,
    COUNT(*) FILTER (WHERE status = 'completed') AS completed_orders,
    COUNT(*) FILTER (WHERE status = 'cancelled') AS cancelled_orders,
    SUM(amount) FILTER (WHERE status = 'completed') AS completed_revenue
FROM orders
```

```

GROUP BY customer_id;

-- Having vs Where
-- WHERE filters rows before aggregation
-- HAVING filters groups after aggregation
SELECT customer_id, COUNT(*) AS orders
FROM orders
WHERE created_at > now() - interval '90 days' -- before aggregation
GROUP BY customer_id
HAVING COUNT(*) > 5 -- after aggregation
ORDER BY orders DESC;

```

Category 3: Self Joins and Recursive CTEs

```

-- Self join: find pairs of employees in the same department
SELECT e1.name AS employee1, e2.name AS employee2, e1.department
FROM employees e1
JOIN employees e2 ON e1.department = e2.department AND e1.id < e2.id;
-- e1.id < e2.id prevents (A,B) and (B,A) both appearing, and avoids (A,A)

-- Recursive CTE: management hierarchy
WITH RECURSIVE hierarchy AS (
    SELECT id, name, manager_id, name AS path, 0 AS depth
    FROM employees WHERE manager_id IS NULL

    UNION ALL

    SELECT e.id, e.name, e.manager_id,
           h.path || ' > ' || e.name, h.depth + 1
    FROM employees e
    JOIN hierarchy h ON e.manager_id = h.id
)
SELECT path, depth FROM hierarchy ORDER BY path;

```

Category 4: Gaps and Islands

This pattern appears frequently in time-series and status-change analysis.

```

-- Find gaps in a sequence of IDs
SELECT expected_id
FROM generate_series(1, (SELECT MAX(id) FROM orders)) AS s(expected_id)
WHERE expected_id NOT IN (SELECT id FROM orders);

-- Find contiguous groups (islands) of active users by date
WITH status_changes AS (
    SELECT
        user_id,
        date,
        status,
        status != LAG(status) OVER (PARTITION BY user_id ORDER BY date) AS is_change
    FROM user_daily_status

```

```

),
groups AS (
  SELECT
    user_id,
    date,
    status,
    COUNT(*) FILTER (WHERE is_change) OVER
      (PARTITION BY user_id ORDER BY date) AS group_id
  FROM status_changes
)
SELECT user_id, status, MIN(date) AS start_date, MAX(date) AS end_date
FROM groups
GROUP BY user_id, status, group_id
ORDER BY user_id, start_date;

```

Category 5: N+1 and Query Efficiency (Backend Roles)

```

-- Bad: N+1 (1 query for orders + N queries for customer names)
-- SELECT * FROM orders;
-- For each order: SELECT name FROM customers WHERE id = ?

-- Good: Single JOIN
SELECT o.id, o.amount, c.name AS customer_name
FROM orders o
JOIN customers c ON o.customer_id = c.id;

-- Lateral join for "first N per group" without N+1
SELECT c.name, recent.id AS recent_order_id, recent.amount
FROM customers c
CROSS JOIN LATERAL (
  SELECT id, amount
  FROM orders
  WHERE customer_id = c.id
  ORDER BY created_at DESC
  LIMIT 3
) AS recent;

```

How to Handle "I Don't Know" in a SQL Interview

Never say "I don't know" and stop. Instead:

1. **Think out loud:** "I'm not sure of the exact syntax, but I think it involves... let me work through the logic first."
2. **Solve the problem in pseudo-SQL:** "I'd use a window function here — something like `ROW_NUMBER()` OVER (PARTITION BY customer_id ORDER BY date DESC) ."
3. **State what you DO know:** "I know I need to partition by customer and order by date descending. The function name escapes me — is it `ROW_NUMBER` or `DENSE_RANK` here?"
4. **Ask for a hint gracefully:** "Would it be okay if I looked at the syntax for the frame clause? I know what I want to accomplish but I'm blanking on the exact keyword."

3. Explaining Query Optimization Under Pressure

The EXPLAIN Walkthrough Framework

When given a slow query and asked to optimize it, use this framework:

1. Get the current plan (60 seconds)

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) <slow query>;
```

2. Identify the bottleneck node (2 minutes) Look for:

- The node with the highest "actual time" contribution
- Nodes where estimated rows << actual rows (planner is wrong → needs ANALYZE)
- Nodes where "Buffers: read" is high relative to "shared hit" (disk I/O bottleneck)
- Sequential Scans on large tables with high selectivity (missing index)
- Nested Loop joins where the inner side is scanned many times

3. State your diagnosis clearly "The bottleneck is the Nested Loop join at line 7. The inner side is doing a sequential scan on `order_items` (~800,000 rows) for each outer row. With 12,000 outer rows, that's 12,000 sequential scans. An index on `order_items.order_id` would convert this to an index scan."

4. Propose and validate the fix

```
-- Proposed fix
CREATE INDEX CONCURRENTLY ON order_items (order_id);

-- Validate: re-run EXPLAIN ANALYZE after creating index
-- Expected: Nested Loop with Index Scan, cost drops from 98,000 to 450
```

Key EXPLAIN Patterns to Know Cold

Pattern 1: High estimated rows, low actual rows → stale statistics

```
Seq Scan on orders (cost=0.00..18000.00 rows=100000 ...) (actual rows=42 ...)
```

Fix: `ANALYZE orders;` or investigate why statistics are wrong (very skewed distribution).

Pattern 2: Index Scan but many "Heap Fetches" → visibility map not populated

```
Index Only Scan on orders (actual rows=5000 Heap Fetches: 4987)
```

Fix: Run `VACUUM orders;` to populate the visibility map. The high Heap Fetches mean the all-visible bits are not set.

Pattern 3: Hash Join with large batches → work_mem too low

```
Hash Join (Batches: 16 Memory Usage: 4096kB)
```

Fix: Increase `work_mem` for this session or globally. The join is spilling to disk.

Pattern 4: Sort with large external sort → work_mem issue

```
Sort (Sort Method: external merge Disk: 45000kB)
```

Fix: `SET work_mem = '128MB';` for this session and re-run. Or add an index on the sort column.

Vocabulary to Use When Discussing Query Performance

Use these terms in interviews to signal depth:

- "The planner's row count estimate is significantly off — this suggests the table needs `ANALYZE`."
 - "The correlation between the column's value and its physical position is low, so PostgreSQL's range estimate for this inequality predicate may be inaccurate."
 - "I'd use a partial index here to reduce the index size and make it fit in `effective_cache_size` more easily."
 - "This join would benefit from a `Hash Join` if there's sufficient `work_mem`, but the current `work_mem` is causing it to fall back to a `Nested Loop`."
 - "The index exists but it's not being used because the cast on the column `created_at::DATE` makes the predicate non-sargable — an expression index would fix this."
-

4. How to Talk About Internals

The MVCC Explanation (Memorize This)

If asked "Explain MVCC" in an interview, here is a production-quality answer at 3 levels of depth:

30-second version (for a quick check): "PostgreSQL stores multiple versions of each row on disk. When you update a row, PostgreSQL writes a new version and marks the old one as dead — it doesn't overwrite in place. Each transaction sees a consistent snapshot of the database that includes all committed changes before the snapshot was taken, and excludes all changes committed after. This means readers never block writers and writers never block readers."

3-minute version (for follow-up questions): "Each row has two system fields: `xmin`, the transaction ID that created it, and `xmax`, the transaction ID that deleted or updated it. When a transaction starts, it takes a snapshot of all currently active transaction IDs. When the transaction reads a row, it applies visibility rules: the row is visible if `xmin` is committed and not in the snapshot (meaning it was committed before the snapshot was taken), AND either `xmax` is zero, or `xmax` is uncommitted, or `xmax` was committed after the snapshot. The result is that each transaction sees a consistent view of the data as of its snapshot time, and concurrent transactions don't block each other. The trade-off is that dead tuples accumulate on disk and must be periodically cleaned by `VACUUM`."

5-minute version (for deep-dive interviews): Add discussion of:

- Transaction ID (xid) as a 32-bit counter (wraparound problem → `VACUUM FREEZE`)
 - The `cmin` and `cmax` system fields (command IDs for within-transaction visibility)
 - Snapshot format: `xmin` (oldest active xid), `xmax` (newest xid+1), and `xip` (list of active xids)
 - How the visibility map and hint bits optimize repeated visibility checks
 - How SSI builds on MVCC to detect serialization anomalies
-

WAL Explanation Framework

The key insight: "PostgreSQL doesn't write data pages to disk before acknowledging a commit. Instead, it writes the WAL record describing the change. Since WAL is sequential, it's fast. On crash recovery, PostgreSQL replays WAL records from the last checkpoint to reconstruct any committed changes."

Why WAL enables replication: "Streaming replication works by streaming these WAL records to standbys in real time. The standby applies the same WAL records, keeping its data pages identical. This is a block-level replica — it must be the same PostgreSQL version, and it inherits all changes including bloat and vacuuming."

Why WAL enables logical decoding: "Logical decoding reads the WAL stream and translates it into row-level changes (INSERT/UPDATE/DELETE). This is how logical replication works — instead of replaying raw WAL, it sends the logical change, which allows cross-version replication and selective table publishing."

VACUUM Explanation Framework

Interviewers love asking about VACUUM because it's a source of many production incidents.

The core explanation: "VACUUM's primary job is to mark dead tuples as reusable space. After an UPDATE or DELETE, the old row version stays on disk with its `xmax` set. VACUUM finds all dead tuples — rows whose `xmax` is a committed transaction that is no longer visible to any active transaction — and marks their space in the Free Space Map as available for new rows."

What VACUUM also does:

1. Updates the Free Space Map (so new rows know where to go)
2. Sets all-visible bits in the Visibility Map (enabling Index Only Scans)
3. Freezes old transaction IDs (preventing the 4-billion-xid wraparound)
4. Updates `pg_class` statistics (`relpages`, `reltuples`) for the query planner

What VACUUM does NOT do: "Regular VACUUM does not return space to the operating system. It only makes space available for future PostgreSQL writes within the same table. VACUUM FULL does return space to the OS but requires an exclusive lock — for production, use `pg_repack` instead."

The Replication Interview Answer

Question: "Walk me through how you would set up a highly available PostgreSQL cluster."

Answer structure: "I'd use Patroni as the HA coordinator, backed by etcd as the distributed configuration store. Here's how it works:

Architecture:

- 3 nodes: 1 primary, 2 standbys
- etcd cluster (3 nodes for quorum, can be separate hosts or co-located)
- PgBouncer on a separate virtual IP for connection routing

How Patroni works: Each node runs a Patroni daemon. The primary holds a 'leader lease' key in etcd with a TTL of 30 seconds. The primary must renew this lease every 10 seconds. If the primary fails to renew — due to crash, network partition, or overload — the lease expires. The standbys compete to acquire the new leader lease. The standby with the most up-to-date WAL wins (or any standby if configured with no lag requirement). Patroni promotes it to primary and demotes the old primary if it comes back.

Failover process:

1. Primary lease expires
2. Standby wins leader election in etcd
3. Patroni calls `pg_ctl promote` on the new primary
4. PgBouncer detects topology change (via `/primary` REST endpoint) and reconnects clients to the new primary
5. Old primary, if it comes back, sees it lost leadership and becomes a standby using `pg_rewind`

What I'd monitor:

- Replication lag via `pg_stat_replication`
 - Patroni REST API health (`/health` , `/leader`)
 - etcd cluster health
 - PgBouncer pool saturation"
-

5. System Design with Databases

The Database System Design Framework

When given a system design question involving a database, structure your answer in 5 phases:

Phase 1 — Requirements (5 minutes) Clarify before designing. Ask:

- What are the read/write ratios? (Analytics = read-heavy; OLTP = mixed)
- What are the latency SLAs? (< 10ms reads? < 100ms writes?)
- What is the data volume? (Rows/day? Total data size? Growth rate?)
- What are the consistency requirements? (Strong consistency needed? Can we tolerate lag?)
- Any compliance/regulatory constraints? (GDPR? SOX? Data residency?)

Phase 2 — Data Model (10 minutes) Sketch the core entities and relationships. Key decisions:

- Normalized vs denormalized (read performance vs write complexity)
- Partitioning strategy for large tables
- Index strategy for critical query patterns
- JSONB vs normalized for variable-schema data

Phase 3 — Scalability (10 minutes) Identify bottlenecks and solutions:

- **Read scaling:** Read replicas (hot standby), read-through caching (Redis)
- **Write scaling:** Partitioning (if sharding not needed), connection pooling
- **Connection scaling:** PgBouncer in front of PostgreSQL
- **When to shard:** Only when single-server write throughput is the bottleneck (very rarely needed for most systems)

Phase 4 — High Availability (10 minutes)

- Patroni + etcd for automatic failover
- Synchronous replication for zero data loss (at cost of latency)
- Asynchronous replication for better performance (at risk of small data loss on failover)
- PITR with pgBackRest for recovery from data corruption/accidental deletion

Phase 5 — Trade-offs (5 minutes) Always discuss trade-offs explicitly:

- "I chose asynchronous replication for performance. The trade-off is a possible 10–30 seconds of data loss on failover. For this billing system, that's not acceptable, so I'd use synchronous replication on the primary write path."
-

Common System Design Scenarios — Database Components

Scenario 1: Design a URL shortener

- Write path: INSERT into `links` table (id, long_url, created_at, click_count)
- Read path: Read-heavy. Use read replicas. Consider Redis cache for hot links.

- Schema: `id` as base62 encoded integer (or ULID), index on `short_code`
- Scale: 100M links → partitioning not needed. 1B+ → range partitioning by `created_at`.
- HA: Patroni with 1 sync standby for zero data loss on writes.

Scenario 2: Design a ride-sharing platform

- Core entities: drivers, riders, trips, locations (time-series), payments
- Hot tables: trips (high insert rate), locations (very high insert rate from drivers)
- Location table: (`driver_id`, `timestamp`, `lat`, `lng`) — partition by timestamp (daily/weekly), BRIN index on timestamp, PostGIS for geographic queries
- Trip matching: geo queries → use PostGIS with GiST index
- Payments: strict consistency → Serializable isolation for balance operations
- Scale: Location data grows ~1M rows/day per 10,000 active drivers. Time-based partitioning with automatic partition creation.

Scenario 3: Design a multi-tenant SaaS

- Multi-tenancy options:
 - **Shared table:** `tenant_id` column, RLS policies. Simplest, hard to migrate one tenant.
 - **Schema per tenant:** `CREATE SCHEMA tenant_42`. Moderate isolation, 10,000+ tenants gets messy.
 - **Database per tenant:** Full isolation, expensive for > 100 tenants.
- For most SaaS: shared tables + RLS + PgBouncer. Schema per tenant for compliance-heavy tenants.
- Indexing: all indexes include `tenant_id` as first column.
- Zero-downtime migrations: challenging with RLS — automate with Flyway/Alembic.

Scenario 4: Design a financial transaction ledger

- Core requirement: Every transaction must be ACID-compliant. No money created or destroyed.
- Schema: double-entry ledger (`debit_account_id`, `credit_account_id`, `amount`, `reference`)
- Consistency: Check constraint: `amount > 0`. Application: sum of all debits = sum of all credits.
- Isolation: Serializable for balance reads + writes to prevent lost updates.
- Auditing: Append-only ledger table + RLS so entries cannot be deleted. `pgaudit` for all DDL.
- HA: Synchronous replication (zero data loss). RPO = 0.

Red Flags in Your System Design

Interviewers notice when candidates:

- Suggest sharding before exhausting single-server optimizations
- Forget to mention connection pooling for a high-concurrency system
- Ignore replication lag when proposing read replicas for consistency-sensitive data
- Design for scale-out before understanding the actual load requirements
- Choose eventual consistency for data that actually requires strong consistency
- Use a UUID as a sharding key without acknowledging hot-spot risk (UUIDs are random, so they distribute well — this is actually fine)

6. Behavioral Questions for Database Engineers

The STAR Format

For all behavioral questions, use STAR: **S**ituation, **T**ask, **A**ction, **R**esult. The result should always include a metric: "This reduced query time from 40 seconds to 200ms" or "This eliminated the production incident class

entirely for 18 months."

Top 15 Behavioral Questions for Database Roles

1. "Tell me about a time you diagnosed and fixed a production database incident." What they're testing: Systematic thinking under pressure, communication, PostMortem culture. *Prepare:* A specific incident. Walk through: detection → diagnosis (tools used, queries run) → root cause → fix → prevention. *Signal:* Mention specific tools (`pg_stat_activity` , `pg_locks` , `EXPLAIN ANALYZE`). Mention the monitoring that caught it. Discuss the postmortem.

2. "Describe a time you had to optimize a slow query." What they're testing: `EXPLAIN` competency, systematic approach, ability to reason about trade-offs. *Prepare:* A real story. Include: the query, the original plan, what you found (missing index, bad statistics, wrong join strategy), the fix, the measured improvement. *Signal:* Use specific vocabulary: "The estimated row count was 5,000 but actual was 12. This suggested stale statistics. After `ANALYZE` , the planner chose an index scan instead of a sequential scan."

3. "Tell me about a database migration you led." What they're testing: Zero-downtime skills, planning ability, risk management. *Prepare:* Include: table size, approach (`CREATE INDEX CONCURRENTLY`, phased column addition), rollback plan, monitoring during migration. *Signal:* "We had a 400M-row table and needed to add a NOT NULL column. Direct `ALTER TABLE` would have locked the table for 15 minutes. Instead, we: added a nullable column with a default, backfilled in 10,000-row batches with a delay between batches to avoid I/O spike, then added the NOT NULL constraint after verifying no NULLs remained — which in PostgreSQL 12+ is a metadata-only operation."

4. "Describe a situation where you disagreed with a teammate's database design decision." What they're testing: Constructive disagreement, data-driven arguments, interpersonal skills. *Prepare:* Focus on the argument you made (with evidence), how you listened to their perspective, and how you reached a decision. *Signal:* "My colleague wanted to denormalize the orders table for faster reads. I ran benchmark queries on a normalized vs denormalized schema with production-sized sample data, and showed that with the right index, the normalized schema was within 5% of the denormalized version's read performance, while having significantly better write performance and simpler migrations."

5. "How have you handled a situation where you had to deliver bad news about database performance?" What they're testing: Honesty, stakeholder communication, problem-solving orientation. *Prepare:* A story where you told a product team their feature couldn't work as designed without a schema change, or where you had to delay a release for a migration. *Signal:* Show that you proposed solutions alongside the bad news. "I couldn't just say 'this won't scale.' I came with three options: (a) redesign the schema (2-week effort), (b) add a caching layer (1-week effort, accepted slightly stale data), or (c) proceed and revisit in 3 months with monitoring."

6. "Tell me about a time you mentored a junior engineer on a database topic." What they're testing: Communication, leadership, ability to explain complex things simply. *Prepare:* A specific explanation you gave — MVCC, index design, `EXPLAIN` reading. Include the impact.

7. "Describe your approach to database capacity planning." What they're testing: Systematic thinking, quantitative reasoning, production experience. *Signal:* Mention: current metrics (IOPS, CPU, connections, disk growth), trend analysis, load testing, planned maintenance, headroom targets.

8. "Tell me about a time a routine change caused an unexpected incident." What they're testing: Intellectual honesty, learning orientation, post-incident process. *Signal:* Be specific about what you thought would happen vs what did happen. Show what you learned.

9. "How do you stay current with PostgreSQL developments?" What they're testing: Genuine interest in the field, intellectual curiosity. *Signal:* Mention specific sources: PostgreSQL release notes, depesz blog, pganalyze blog, Cybertec blog, PGCon talks. Mention a specific recent feature you found interesting (e.g., PG 16's logical replication improvements, PG 17 improvements).

10. "Describe how you would approach a database schema review for a new project." What they're testing: Systematic thinking, depth of knowledge. *Signal:* Mention: normalization check, constraint completeness, index strategy for stated query patterns, naming conventions, migration safety, VACUUM considerations for high-update tables.

11. "Tell me about a time you had to balance performance with data integrity." What they're testing: Judgment, pragmatism, understanding of trade-offs. *Prepare:* A story where you chose a slightly weaker isolation level for performance, or chose to denormalize with an agreed data synchronization strategy.

12. "How do you approach monitoring and alerting for a PostgreSQL database?" What they're testing: Production readiness, observability knowledge. *Signal:* Mention: `pg_stat_statements` for query performance, `pg_stat_replication` for replication lag, `pg_stat_user_tables` for autovacuum health, `pg_locks` for lock monitoring, custom alerts for bloat, long-running transactions, connection pool saturation.

13. "Describe a time you introduced a new database tool or practice to your team." What they're testing: Initiative, persuasion, change management. *Prepare:* PgBouncer adoption, `pg_stat_statements` setup, pgBackRest migration, or similar.

14. "Tell me about the most complex database schema you've designed." What they're testing: Depth of schema design experience. *Prepare:* Walk through the business requirements, the key entities, the normalization decisions, the indexing strategy, any partitioning, and lessons learned.

15. "What's the biggest PostgreSQL mistake you've made, and what did you learn?" What they're testing: Self-awareness, learning culture, honesty. *Prepare:* An honest story. Common examples: ran VACUUM FULL in production during peak hours, created an index on the wrong column, didn't test a migration on production-sized data.

7. Level-Specific Question Banks

Junior Level (L3 / Junior Engineer)

These questions test SQL basics and basic PostgreSQL knowledge.

1. What is a JOIN? Explain the difference between INNER, LEFT, and FULL OUTER JOIN.
 2. What is the difference between WHERE and HAVING?
 3. Can you explain what a NULL value is, and how it affects comparisons?
 4. What is an index, and why would you add one?
 5. What is a transaction, and what happens if a transaction is not committed?
 6. What is the difference between `DELETE` and `TRUNCATE` ?
 7. Write a query to find the second-highest salary in an employees table.
 8. What is a subquery, and when would you use one vs a JOIN?
 9. What is a primary key, and what makes it different from a unique constraint?
 10. What is the difference between `UNION` and `UNION ALL` ?
-

Mid Level (L4 / Software Engineer II)

These questions test query fluency and basic PostgreSQL proficiency.

1. What is a window function? Give an example of when you'd use `RANK()` vs `ROW_NUMBER()`.
 2. Explain what a CTE is and when you'd use one instead of a subquery.
 3. What is the difference between `CHAR`, `VARCHAR`, and `TEXT` in PostgreSQL?
 4. When would you use a GIN index vs a B-Tree index?
 5. What is `EXPLAIN ANALYZE`, and what does it tell you?
 6. What does the `COALESCE` function do?
 7. Explain the difference between `=` and `IS` for NULL comparisons.
 8. What is a `SEQUENCE` in PostgreSQL? How does it differ from `SERIAL`?
 9. Write a query to find duplicate rows in a table.
 10. What is MVCC? Explain it in 2 minutes.
 11. What is the difference between `Read Committed` and `Repeatable Read` isolation?
 12. What is `SELECT FOR UPDATE`, and why would you use it?
 13. What is bloat, and what causes it?
 14. Explain the difference between a partial index and an expression index.
 15. What is the `LATERAL` keyword?
-

Senior Level (L5 / Senior Engineer)

These questions test deep understanding of PostgreSQL internals and production experience.

1. Explain MVCC in detail, including xmin, xmax, and how snapshots work.
 2. What is WAL, and why does it enable both crash recovery and replication?
 3. Walk me through what happens, step by step, when you run `UPDATE orders SET status = 'shipped' WHERE id = 1`.
 4. Explain what VACUUM does and why it's necessary.
 5. What is the difference between a replication slot and WAL archiving?
 6. How does streaming replication differ from logical replication?
 7. You have a 500M-row table and need to add an index. How do you do it without downtime?
 8. Explain the difference between `work_mem`, `shared_buffers`, and `effective_cache_size`.
 9. What is a checkpoint, and what are the relevant configuration parameters?
 10. How do you diagnose and resolve deadlocks in PostgreSQL?
 11. What is TOAST, and when does it come into play?
 12. Explain row-level security. How would you implement multi-tenant isolation?
 13. A query that used to take 10ms now takes 10 seconds. Walk me through your investigation.
 14. What is autovacuum? How would you tune it for a table with a very high update rate?
 15. Explain the visibility map and how it enables Index Only Scans.
 16. What is `pg_repack`, and when would you use it instead of `VACUUM FULL`?
 17. What is the wraparound problem in PostgreSQL, and how does VACUUM FREEZE help?
 18. How would you implement a distributed lock using PostgreSQL?
 19. What is the difference between `synchronous_commit = on` and `synchronous_commit = remote_apply`?
 20. How would you set up PITR for a critical PostgreSQL database?
-

Staff Level (L6+ / Staff Engineer / Principal)

These questions test architecture, trade-offs, and leadership depth.

1. Design a high-availability PostgreSQL architecture for a system that cannot afford more than 30 seconds of downtime and less than 1MB of data loss.
2. How would you approach a major version upgrade of a PostgreSQL cluster with zero downtime?

3. A table has 10 billion rows, and a new feature requires adding a column with a unique constraint. Walk through your approach.
 4. Your PostgreSQL cluster is at 80% CPU. What are your investigation steps, and what are the possible solutions?
 5. Describe the trade-offs between schema-per-tenant, table-per-tenant, and shared-table multi-tenancy in PostgreSQL.
 6. How would you implement Change Data Capture (CDC) using PostgreSQL, and what are the reliability guarantees?
 7. A team wants to shard their PostgreSQL database. What questions would you ask, and what would likely change their mind?
 8. Walk me through designing a backup and disaster recovery strategy that meets RPO=1 minute and RTO=15 minutes.
 9. How would you approach migrating from PostgreSQL 14 to PostgreSQL 16 with a 5TB database and no maintenance window?
 10. Explain the trade-offs between synchronous and asynchronous replication in terms of data loss, latency, and throughput.
 11. How do you explain to a product team why a feature they want would cause database performance problems?
 12. What is Serializable Snapshot Isolation? When would you use it, and what are the application-level implications?
 13. How would you architect a PostgreSQL-based system to comply with GDPR "right to erasure" requirements?
 14. Walk me through the Patroni failover process in detail, including what happens to in-flight transactions.
 15. You inherit a PostgreSQL database with autovacuum effectively disabled, 40% bloat on the main table, and replication lag of 30 seconds. What is your recovery plan?
-

8. The Salary Negotiation Playbook

Principles

1. **Never give the first number.** Say: "I'm more interested in understanding the full scope of the role. Could you share the budgeted range?"
2. **Always negotiate.** 80% of hiring managers expect it. Not negotiating is leaving money on the table.
3. **Negotiate the full package.** Base salary, equity (stock options/RSUs), signing bonus, performance bonus, PTO, remote flexibility, professional development budget.
4. **Get the offer in writing before evaluating.** Verbal offers can be rescinded or "misremembered."
5. **Give yourself 24–48 hours.** "Thank you so much for the offer. I'm genuinely excited about this opportunity. Could I have until [specific date] to review the details?"

Database Engineer Compensation Benchmarks (2025, US)

These are approximate ranges. Adjust for location (SF/NYC pay 20–30% more), company size, and level.

Level	Title	Base (USD)	Total Comp (USD)
L3	Junior DB Engineer	\$100K–\$130K	\$110K–\$145K
L4	DB Engineer / Backend Engineer	\$130K–\$165K	\$150K–\$220K
L5	Senior DB Engineer	\$165K–\$210K	\$220K–\$350K

L6	Staff DB Engineer	\$200K–\$250K	\$320K–\$500K
L7+	Principal / Distinguished	\$250K–\$300K+	\$500K–\$1M+

Note: FAANG total comp is heavily equity-weighted. Startup offers may have higher base but lower/riskier equity.

Negotiation Scripts

Countering the low offer: "I really appreciate the offer. Based on my research on market rates for this level and the scope you've described — particularly the [specific aspect: e.g., ownership of the entire data infrastructure] — I was expecting something closer to [\$X]. Is there flexibility there?"

When they ask for your current salary: "I'm focused on the market rate for this role and my contributions here. Based on what I've learned about the role, I believe [\$X–Y] range is fair."

Leveraging a competing offer: "I want to be transparent — I do have another offer at [\$X]. I'm most excited about this role because of [specific reason], so I wanted to give you the opportunity to match before I decide."

Negotiating equity: "The base looks good. I'd like to understand the equity better — could you share the current 409A valuation, total shares outstanding, and the company's last fundraising valuation? That helps me evaluate the RSUs."

Getting a signing bonus: "The base and equity look good. To bridge the gap on unvested equity I'd be leaving behind, would you be able to include a [\$X] signing bonus?"

Red Flags in an Offer

- No equity for a senior+ role at a funded startup
- Equity with no cliff (suggests poor retention culture)
- Non-compete that is unusually broad
- "At-will but we expect 12+ months" — that's just at-will
- Verbal offer that takes 2+ weeks to produce written documentation
- "We don't negotiate" as the first response — this is a negotiating tactic, not a policy

9. Company-Specific Preparation

Amazon (AWS / Amazon Data Services)

Interview format:

- 4–6 rounds
- Strong emphasis on Leadership Principles in behavioral rounds (LP stories required)
- SQL round often involves a take-home or live coding in a shared document
- Bar Raiser round: focuses on leadership principles and broad technical depth

Technical focus:

- Window functions and complex aggregations
- Schema design for high-scale OLTP systems
- DynamoDB vs PostgreSQL trade-offs (know when each is appropriate)
- Query optimization and EXPLAIN fluency
- Aurora PostgreSQL-specific features

Leadership Principles to highlight:

- Dive Deep (goes with database internals)
- Insist on Highest Standards (goes with schema review and migration safety)
- Bias for Action (quick diagnosis and resolution of incidents)
- Ownership (I owned the entire database infrastructure, not just my ticket)

Prep resources: [23_Company_Preparation/amazon.md](#)

Google (Google Cloud Spanner, AlloyDB, Big Query)

Interview format:

- 4–5 rounds
- "Googlyness" round (behavioral)
- General coding round may include SQL or database-adjacent algorithm problems
- System design round with strong emphasis on scale

Technical focus:

- Distributed database concepts (they use Spanner internally)
- BigQuery and analytical SQL patterns
- Scale-first thinking (10x, 100x scale questions)
- Trade-offs between consistency and availability

Differentiate yourself: Know that Google uses Spanner for global ACID transactions. Understanding the difference between Spanner's TrueTime approach vs PostgreSQL's MVCC approach is a strong signal.

Prep resources: [23_Company_Preparation/google.md](#)

Microsoft (Azure Database for PostgreSQL, Azure Cosmos DB)

Interview format:

- 4–5 rounds, including a coding round (often LeetCode-style) and system design
- Behavioral: growth mindset, collaboration, customer obsession

Technical focus:

- Azure Database for PostgreSQL Flexible Server vs Cosmos DB
- Hyperscale (Citus) for distributed PostgreSQL workloads
- Geo-replication and data residency for enterprise customers
- .NET / C# integration patterns with PostgreSQL (if backend role)

Prep resources: [23_Company_Preparation/microsoft.md](#)

Meta (Facebook)

Interview format:

- 5–6 rounds
- Coding round: typically LeetCode-style, not SQL-specific, but DB design questions appear
- System design: Emphasis on feed ranking, social graph, and messages — all data-heavy

Technical focus:

- Social graph data models (graph databases vs relational approaches)
- MySQL is Meta's primary DB (know it's different from PostgreSQL but concepts transfer)

- Large-scale data warehousing (Presto/Trino on top of HDFS/S3)
- Messaging systems (TAO, MQ, ZippyDB)

Note: Meta primarily uses MySQL, not PostgreSQL. Many PostgreSQL concepts transfer, but demonstrate MySQL awareness if targeting Meta.

Prep resources: `23_Company_Preparation/meta.md`

Uber

Interview format:

- 3–4 rounds, focused on practical engineering
- System design round is common for data/backend roles
- May include debugging a slow query or a replication setup

Technical focus:

- Schemaless (Uber's custom MySQL-based storage) — understand the concept
- Real-time data processing: location data at extreme scale
- Geospatial queries (H3 spatial indexing, PostGIS)
- Database migrations at high traffic (they famously migrated MySQL → PostgreSQL → MySQL)

Known interview problems:

- Design the database for a ride-sharing system
- How would you handle geospatial queries at scale?
- Explain how you'd implement surge pricing with correct ACID semantics

Prep resources: `23_Company_Preparation/uber.md`

Netflix

Interview format:

- 4–5 rounds, heavily focused on ownership and systems thinking
- "Keeper Test" culture: high bar for self-direction
- System design with emphasis on global availability and resilience

Technical focus:

- Global data distribution (multi-region PostgreSQL or Cassandra)
- Read/write patterns for streaming services (playback state, recommendations)
- Data engineering at scale (Apache Iceberg, Spark, Flink integration)
- Chaos engineering perspective: how does your DB design survive failures?

Differentiate yourself: Netflix has written extensively about their database practices (Hollow, EVCache, their migration to CockroachDB for some workloads). Familiarity with their engineering blog signals genuine interest.

Known emphasis:

- Global availability > strong consistency for most Netflix use cases
- PostgreSQL for internal tooling, control plane databases
- Large-scale batch processing over read replicas

Prep resources: `23_Company_Preparation/netflix.md`

Shopify

Interview format:

- Practical, product-focused engineering culture
- System design with real e-commerce constraints
- Strong emphasis on Rails + PostgreSQL (Shopify uses Rails extensively)

Technical focus:

- Multi-tenant at extreme scale (millions of merchants on shared infrastructure)
- MySQL and PostgreSQL fluency (Shopify uses both)
- Database migrations at scale without downtime
- Read replica routing and connection pooling
- Inventory, orders, and checkout — high-consistency workloads

Known emphasis:

- Zero-downtime deployment culture (migrations must be backward-compatible)
- They've written extensively about their database scaling challenges (Pods architecture)

Prep resources: `23_Company_Preparation/shopify.md`

Stripe

Interview format:

- 3–4 rounds, very focused on code quality and design clarity
- Database design is prominent in system design rounds (payments require correctness)

Technical focus:

- Idempotency keys: database design for safe retries
- Financial correctness: double-entry bookkeeping, ACID guarantees
- Event sourcing and append-only ledger patterns
- PostgreSQL for internal services (Stripe uses PostgreSQL extensively)

Known interview themes:

- Design an idempotent payment API
- How would you ensure a charge cannot be applied twice?
- Design the schema for a multi-currency financial system

Prep resources: `23_Company_Preparation/stripe.md`

10. The Day-Before Checklist

Technical Review

- ☐ Can explain MVCC without notes (2-minute version)
- ☐ Can explain WAL and why it matters for replication
- ☐ Can read a 15-node EXPLAIN ANALYZE plan
- ☐ Can write a window function query in < 2 minutes
- ☐ Can write a recursive CTE from memory
- ☐ Know the index selection rule (B-Tree vs GIN vs BRIN vs Partial)

- ☐ Can explain deadlock detection and prevention
- ☐ Can describe Patroni failover step by step
- ☐ Have a system design template memorized (Requirements → Model → Scale → HA → Trade-offs)

Logistics

- ☐ Test your video setup (camera, microphone, lighting)
- ☐ Test screen sharing in the platform being used (Zoom, Google Meet, CoderPad)
- ☐ Have a good internet connection (use ethernet if possible)
- ☐ Have a backup plan for internet failure (phone hotspot)
- ☐ Know your interviewer's names (check LinkedIn, be prepared to ask if you forget)
- ☐ Have your résumé open in a separate tab
- ☐ Have blank paper and a pen for system design diagrams

Mental Preparation

- ☐ Review your top 5 behavioral stories (STAR format, specific metrics)
- ☐ Write out your "Tell me about yourself" in < 90 seconds
- ☐ Prepare 3 thoughtful questions to ask each interviewer
- ☐ Review the company's recent engineering blog posts
- ☐ Sleep at least 7 hours

Questions to Ask Your Interviewers

At the end of every round, you should ask 1–2 questions. Have 8–10 prepared so you don't repeat.

Technical questions:

- "What does the database infrastructure look like today, and where do you see the biggest challenges in the next 12 months?"
- "How does the team handle database migrations — what's the process and who owns it?"
- "What monitoring and observability do you have for the database layer today?"
- "Has there been a significant database incident recently? What caused it and what changed after?"

Culture questions:

- "What's the on-call rotation like for database incidents?"
- "How does the database engineering team interface with product engineers?"
- "What does success look like for someone in this role in the first 6 months?"
- "What do you like most about working on the database infrastructure here?"

11. Post-Interview Protocol

Immediately After Each Round (Within 1 Hour)

Write down:

- The questions you were asked
- Your answers (what went well, what didn't)
- Technical topics where you felt unsure
- Names of your interviewers and their specific follow-up questions

This information is valuable for: (a) your debrief/follow-up email, (b) studying gaps before subsequent rounds, and (c) preparing for future interviews at other companies.

Thank-You Email (Within 24 Hours)

Send a brief, personalized email to your recruiter. If you got your interviewers' emails, send individual notes.

Template: "Hi [Name], thank you for taking the time to speak with me today. I enjoyed our discussion about [specific topic from the conversation]. [One sentence showing you retained something specific from the interview — technical or personal.] I remain very excited about the opportunity. Please let me know if there's anything else I can provide. Best, [Your Name]"

If You Don't Hear Back in the Stated Timeframe

Follow up with your recruiter after 5 business days from when they said you'd hear back. Keep it brief and positive: "Hi [Name], I wanted to follow up on the status of the [Role] position. I remain very interested. Is there an update on timing?"

Handling a Rejection

Ask for feedback: "Thank you for letting me know. I understand. If possible, I'd really appreciate any feedback on my interview performance — I'm committed to improving, and any specific areas you can share would be genuinely valuable."

Rejections are data points. Log the questions you couldn't answer. Study those areas. Reapply in 6–12 months.

Handling Multiple Offers

Be transparent (but not overly revealing) with each company when you have a deadline: "I have another offer with a deadline of [date]. I'd prefer to join [your company] if we can align on the details by then. Is that possible?"

This creates urgency without being adversarial.

See [19_Interview_Questions](#) for the full question bank organized by topic and difficulty. See [20_Interview_Tasks](#) for hands-on practice problems. See [23_Company_Preparation](#) for company-specific interview guides. See [24_Cheat_Sheets](#) for quick-reference materials to review the day before.